MyAccounting — Functional Analysis

Document version: 1.0
Application version: 1.0.0
Date: April 14, 2026
Author: Daniel Seguin

Table of Contents

1. Purpose and Scope
2. System Overview
3. Architecture
4. User Authentication and Access Control
5. Dashboard
6. Configuration Module
7. Invoicing Module
8. Cash Receipts Module
9. General Journal Module
10. Chart of Accounts (G/L) Module
11. Financial Reports Module
12. Timesheet Module
13. Data Import
14. Database Design
15. Business Rules and Validations
16. Integrations and Output Formats
17. Build and Deployment
18. Security Considerations

---

1. Purpose and Scope

MyAccounting is a desktop accounting application designed for small Canadian service-industry businesses. It covers the complete accounts-receivable (AR) cycle and general ledger workflow for a single company on a single SQL Server instance.

In scope:

- User login and role-based access control
- Customer, inventory, project, and lookup data management
- Invoice creation, PDF generation, and email delivery
- Cash receipt recording and G/L posting
- General journal entry (double-entry bookkeeping)
- Chart of Accounts maintenance and transaction drill-down
- Trial Balance, Income Statement, and Balance Sheet reporting
- Consultant timesheet management, approval workflow, PDF print, and client email
- One-time seed-data import from flat files

Out of scope:

- Accounts payable (vendor invoices and payments)
- Payroll processing
- Inventory quantity tracking (items are service-based)
- Multi-company or multi-currency consolidation
- Cloud or web-based deployment

---

2. System Overview

Attribute	Value
Platform	Windows desktop (standalone executable or Python runtime)
UI framework	Python `tkinter` + `ttk`
Database	Microsoft SQL Server (local), database `myaccounting`
DB driver	ODBC Driver 17 for SQL Server, Windows Authentication
Language	Python 3.9+
Key libraries	`pyodbc`, `reportlab`, `openpyxl`, `tkcalendar`
Entry point	`app.py`

The application is a single-process GUI. All data access goes through `db.py`, which maintains a stateless connection model (one connection opened and closed per operation). There is no application server or REST API layer.

---

3. Architecture

```
┌──────────────────────────────────────────────────────────┐
│                       app.py (Main Window)                 │
│  ┌──────────┐  ┌──────────────────────────────────────┐  │
│  │ Sidebar  │  │              Content Frame            │  │
│  │ Nav menu │  │  Dashboard / Module panels loaded here  │  │
│  └──────────┘  └──────────────────────────────────────┘  │
└──────────────────────────────────────────────────────────┘
          │
          │  imports / instantiates
          ▼
┌──────────────────────────────────────────────────────────┐
│  Module Layer                                             │
│  auth.py  login_dialog.py  config_module.py               │
│  invoice_module.py  cash_receipt_module.py  journal_module.py │
│  gl_module.py  reports_module.py  timesheet_module.py     │
└──────────────────────────────────────────────────────────┘
                      │
                      │  all DB calls
                      ▼
              ┌──────────────┐
              │    db.py     │   pyodbc → SQL Server
              └──────────────┘
                      │
                      ▼
          ┌──────────────────────┐
          │  SQL Server          │
          │  database: myaccounting│
          └──────────────────────┘
```

Module Responsibilities

File	Responsibility
`app.py`	Application entry point. Builds the main `tk.Tk` window, sidebar navigation, dashboard, About dialog, and Support Request dialog. Orchestrates module loading into the content frame.
`auth.py`	Password hashing (PBKDF2-HMAC SHA-256) and credential verification. Module access lookup.
`login_dialog.py`	Modal login dialog displayed at startup. Exits the application if cancelled.
`db.py`	All database interaction: table creation/migration, settings management, posting logic, G/L summary refresh, invoice attachment helpers, and sequence (auto-number) management.
`config_module.py`	Configuration UI. 13 tabs covering company info, fiscal year settings, all lookup tables, email settings, and user management.
`invoice_module.py`	Invoice creation, editing, PDF generation, and email delivery.
`cash_receipt_module.py`	Cash receipt entry, verification listing, and G/L posting.
`journal_module.py`	General journal entry creation, editing, verification listing, and posting.
`gl_module.py`	Chart of Accounts list, account CRUD, transaction drill-down, and export.
`reports_module.py`	Trial Balance, Income Statement, Balance Sheet; PDF and Excel export.
`timesheet_module.py`	Full timesheet lifecycle: create/edit/delete, work items per day, status workflow, PDF print, email to client, and TPG CSV/PDF/Excel import.
`import_data.py`	One-time bootstrap: reads flat files from `importdir/` and seeds SQL Server tables.
`utils.py`	Shared helpers: `fmt_currency`, `fmt_date`, `parse_amount`, `parse_date`, `today_str`, `make_date_field`.

4. User Authentication and Access Control

4.1 Login Flow

1. On startup, `app.py` instantiates `LoginDialog` (modal).
2. The user enters a username and password.
3. `auth.authenticate_user()` queries the `users` table. If the account is inactive (`is_active = 0`) or not found, login is refused.
4. The stored password is validated with `hmac.compare_digest` against the PBKDF2-HMAC SHA-256 hash.
5. On success, a user context dict (`user_id`, `username`, `userfullname`, `email`, `role`) is stored in the main application instance.
6. If the dialog is cancelled, the application exits immediately.

4.2 Password Security

Parameter	Value
Algorithm	PBKDF2-HMAC SHA-256
Iterations	260,000
Salt	16 bytes, randomly generated per password (`os.urandom`)
Storage format	`<salt_hex>:<hash_hex>` in the `users.password` column
Comparison	`hmac.compare_digest` (constant-time, prevents timing attacks)

4.3 Roles

Role	Description
`admin`	Full access to all modules. Bypasses the `useraccess` table entirely.
`user` / `accountant`	Per-module access level defined in the `useraccess` table.

4.4 Module Access Levels

Each non-admin user has an explicit access level per module stored in the `useraccess` table:

Level	Effect
`full`	Can read and write; all action buttons enabled.
`view only`	Module opens in read-only mode; write action buttons are disabled.

Modules covered by access control: GL Chart of Accounts, Journal Entry, Reports, Configuration, Invoicing, Cash Receipts, Timesheets.

5. Dashboard

The Dashboard is the default home screen, loaded automatically after login.

5.1 Panels

Panel	Data Source	Description
AR Balances	`invoicemaster`	Outstanding receivables by customer. Displays customer name and balance due.
In-Process Journals	`journalmaster`	Count of journal entries with status `Draft` (not yet posted).
Revenue Chart	`glsummary`	Bar chart of monthly revenue from the revenue G/L account. Rendered in a `tk.Canvas`.
To-Do List	`todos` table	Personal task list. Supports add, mark-complete, and delete. Tasks have a title, notes, due date, and priority (`Low`, `Normal`, `High`).

5.2 To-Do List

- Tasks are stored per-user in the `todos` table (`user_id`, `title`, `notes`, `due_date`, `priority`, `status`, `created_date`).
- Status values: `Open`, `Done`.
- The list is filtered to show only the logged-in user's tasks.

6. Configuration Module

The Configuration module (`config_module.py`) is the administrative hub. It is organised into 13 tabs.

6.1 Tab Summary

#	Tab	Contents
1	Company Info	Company name, address lines, city, province, postal code, country, phone, fax, email, website, logo path
2	Fiscal Year & Settings	Fiscal year start/end dates, tax rate (%), currency, invoice prefix, journal prefix, receipt prefix, G/L account defaults (AR, Revenue, Tax Payable, Bank/Cash)
3	Customers	Full CRUD on the `customers` table. Customer dialog includes billing/shipping addresses, salesman, tax code, payment terms, credit limit, revenue account, manager contact, language, and notes.
4	Inventory	Full CRUD on the `inventory` table. Item code, description, unit, unit price, G/L revenue account.
5	Projects	Full CRUD on the `projects` table. Project name and description.
6	Salesmen	Full CRUD on the `salesmen` table. Salesman name and email.
7	Tax Codes	Full CRUD on the `taxcodes` table. Code, description, rate (%).
8	Countries	Full CRUD on the `country` table. Country code, name, default currency.
9	Provinces	Full CRUD on the `provinces` table. Country code, province code, province name.
10	Payment Terms	Full CRUD on the `payment_terms` table. Term name and number of days.
11	Email Settings	SMTP host, port, username, password, TLS flag, support destination email.
12	Users	Add/edit users, reset passwords, assign per-module access levels. Admin-only tab.
13	Timesheet Settings	Default shift type (`8to4` , `9to5`), approval-required flag.

6.2 Email Welcome Message

When a new user is created, the system sends a welcome email to the new user's address containing their username, a temporary password, and login instructions. This uses the SMTP settings from the `settings` table.

---

7. Invoicing Module

7.1 Features

- List all invoices in a sortable `Treeview` (invoice number, date, customer, total, status).
- Create, edit, and delete invoices.
- Invoice status lifecycle: `Draft` → `Approved` → `Void`.
- Add line items from the `inventory` table (item code, description, quantity, unit price, extended amount).
- Support for text-only separator lines (`line_type = 'text'`) in addition to standard item lines.
- Automatic HST calculation using the `tax_rate` setting.
- Optional `comments` field on the invoice header.
- Generate and preview a professional PDF invoice using ReportLab.
- Email the PDF invoice to the customer's billing email via SMTP.
- Manage file attachments per invoice (`invoice_attachments` table).

7.2 Invoice Numbering

- Auto-number derived from the `next_invoice_number` setting plus the `invoice_prefix` (default `INV-`).
- The sequence is incremented in the `settings` table each time a new invoice is saved.

7.3 Invoice Tables

Table	Key Columns
`invoicemaster`	`invoice_id` , `invoice_number` , `invoice_date` , `customer_id` , `salesman_id` , `project_id` , `subtotal` , `tax_amount` , `total` , `status` , `comments`
`invoicedetails`	`line_id` , `invoice_id` , `item_id` , `description` , `qty` , `unit_price` , `amount` , `line_type`
`invoice_attachments`	`attachment_id` , `invoice_id` , `filename` , `filepath`

7.4 PDF Invoice Layout

The PDF is produced by ReportLab (`reportlab.platypus`). The layout includes:

- Company name, address, phone, and email in the header
- Optional company logo (configured in Company Info)
- Invoice number, date, customer details, and salesman
- Line-item table with description, quantity, unit price, and amount
- Subtotal, HST amount, and total
- Comments block at the bottom

7.5 Email Delivery

The invoice email subject and body are template strings stored in `settings` with placeholders: `{invoice_number}`, `{customer}`, `{total}`, `{consultant}`, `{company}`, `{email}`, `{phone}`.

---

8. Cash Receipts Module

8.1 Features

- List all cash receipts in a `Treeview` (receipt number, date, customer, amount, status).
- Create, edit, and delete cash receipt records.
- Status lifecycle: `Draft` → `Posted`.
- Select the customer and optionally link the receipt to a specific open invoice.
- Specify bank/cash G/L account, AR account, and memo.
- Verification listing: text preview of the proposed G/L entries before posting.
- **Post** action: writes two lines to `gldetails` (DR Bank, CR AR) and updates `glsummary`.

8.2 Cash Receipt Numbering

- Auto-number from `next_receipt_number` + `receipt_prefix` (default `CR-`).

8.3 Posting Logic

On post, `db.py` inserts two `gldetails` rows:

Entry	Account	Side	Amount
Debit	Bank/Cash account (`cash_account` setting)	DR	Receipt amount
Credit	Accounts Receivable account (`ar_account` setting)	CR	Receipt amount

The `glsummary` table is refreshed after posting (aggregated debits/credits per account per period).

---

9. General Journal Module

9.1 Features

- List all journal entries (`Treeview` with entry number, date, description, reference, status).
- Create, edit, and delete `Draft` entries. Posted entries are read-only.
- Add, edit, and remove detail lines (account, description, debit, credit).
- Real-time running debit/credit totals with balance validation.
- Verification listing: formatted text preview of the journal entry before posting.
- **Post** action: inserts all detail lines to `gldetails` and refreshes `glsummary`.
- Export journal entries to CSV.

9.2 Journal Numbering

- Auto-number from `next_journal_number` + journal prefix (default no prefix).

9.3 Validation Rules

- Total debits must equal total credits before a Draft entry can be posted (balanced entry).
- At least two detail lines are required.
- Account number must exist in `glmaster`.

9.4 Journal Tables

Table	Key Columns
`journalmaster`	`entry_id`, `entry_number`, `entry_date`, `description`, `reference`, `status`, `created_date`, `posted_date`
`journaldetails`	`line_id`, `entry_id`, `account_number`, `description`, `debit`, `credit`

---

10. Chart of Accounts (G/L) Module

10.1 Features

- List all accounts from `glmaster` in a sortable `Treeview` (account number, name, type, balance type, opening balance, net balance).
- Add, edit, and delete accounts.
- Select any account and view its full transaction history from `gldetails`.
- Export the Chart of Accounts to **CSV**, **Excel** (`.xlsx`), or **PDF**.

10.2 Account Structure

Column	Description
`account_number`	Alphanumeric identifier (e.g., `1060`, `4000`)
`account_name`	Display name
`account_type`	Category (e.g., Asset, Liability, Revenue, Expense)
`balance_type`	`Dr` (normal debit balance) or `Cr` (normal credit balance)
`opening_balance`	Starting balance imported from Sage 50 export

10.3 Net Balance Calculation

Net balance = `opening_balance + Σ debits − Σ credits` for Dr-type accounts, or `opening_balance + Σ credits − Σ debits` for Cr-type accounts. The calculation is always live from `gldetails` — the `glsummary` table is a cached aggregate used only for report performance.

---

11. Financial Reports Module

11.1 Available Reports

Report	Account Range	Description
Trial Balance	All accounts	Lists every account with its net debit or credit balance. Total debits must equal total credits.
Income Statement	Accounts 4000–9999	Revenue minus expenses for the selected date range. Shows net income / net loss.
Balance Sheet	Accounts 1000–3999	Assets, liabilities, and equity as at a selected date. Shows total assets vs. total liabilities + equity.

11.2 Date Filters

- Fiscal year start and end dates default from the `settings` table.
- The user can override the date range in the report parameters panel before generating.

11.3 Export Formats

Format	Library	Notes
PDF	`reportlab`	Formatted multi-page report with company header, page numbers, and totals
Excel (`.xlsx`)	`openpyxl`	Styled workbook with bold headers, column widths, currency formatting
On-screen	`ttk.Treeview` + `ScrolledText`	Interactive view inside the module panel

---

12. Timesheet Module

12.1 Overview

The Timesheet module manages billable-hours tracking for consultants against client engagements. It supports a per-month, per-consultant master record with daily detail rows and multiple work items per day.

12.2 Features

Feature	Description
List view	Filter timesheets by year, month, and consultant name
Create	New timesheet bound to a salesman, year/month, client (`customers` table), and project
Edit	Update header fields and all daily detail rows inline
Delete	Remove a Draft timesheet and all its detail rows (cascade delete)
Work items	Each day row has 1–N work-item lines with a description, Azure DevOps work-item reference, start/end times (AM, PM, Other), and auto-calculated hours
Time parsing	Accepts `HH:MM`, `HHMM`, or `H:MM`; normalises to `HH:MM`
Status workflow	`Draft` → `Submitted` → `Approved` / `Rejected`
PDF print	Professional landscape timesheet layout with company header, consultant info, daily grid, and total hours (ReportLab)
Email to client	Sends the PDF as an attachment to the client manager's email (`customers.manager_email`). Body includes a configurable approval-request message when `timesheet_require_approval = 1`.
Excel export	Exports detail to a formatted `.xlsx` workbook (requires `openpyxl`)
TPG import	Parses a TPG-format file (CSV, PDF via `pdfplumber` / `pypdfium2` /OCR fallback) and previews rows in a confirmation dialog before importing

12.3 Status Workflow

```
Draft  ──[Submit]──►  Submitted  ──[Approve]──►  Approved
                                 ──[Reject] ──►  Rejected

Rejected  ──[Reset to Draft]──►  Draft
Approved  ──[Read-only; cannot be edited]
```

12.4 Timesheet Database Tables

Table	Key Columns	Notes
`timesheetmaster`	`ts_id`, `salesman_id`, `ts_year`, `ts_month`, `client`, `project`, `customer_id`, `status`, `notes`, `created_date`	Unique per `(salesman_id, ts_year, ts_month)`
`timesheetdetail`	`line_id`, `ts_id`, `work_date`, `am_start`, `am_end`, `pm_start`, `pm_end`, `other_start`, `other_end`, `total_hours`	One row per working day; cascade deletes with master
`timesheetworkitem`	`item_id`, `line_id`, `sort_order`, `description`, `devops_work_item`, `hours`	One row per work item per day; cascade deletes with detail

12.5 Hour Calculation Rules

- Hours per work item = `end_time - start_time` (decimal hours, rounded to 2 decimal places).
- Daily `total_hours` = sum of all work-item `hours` for that day.
- Monthly total = sum of all `timesheetdetail.total_hours` for the timesheet.
- Invalid or empty time values produce 0 hours; the UI shows a warning but does not block saving.

12.6 TPG Timesheet Import

Supported source formats:

Format	Parser	Fallback
CSV	Native Python `csv` module	—
PDF (text-based)	`pdfplumber`	`pypdfium2` renderer
PDF (scanned/image)	`easyocr` OCR	—
Excel (`.xlsx`)	`openpyxl`	—

After parsing, a preview dialog shows the interpreted rows. The user can confirm or cancel. On confirm, a new `timesheetmaster` record and associated `timesheetdetail` + `timesheetworkitem` rows are inserted.

---

13. Data Import

`import_data.py` is a standalone bootstrap script run **once** to populate the database from flat files in `importdir/`.

13.1 Import Files

File	Table(s) Populated	Format
`customers.txt`	`customers`	Pipe-delimited or fixed-width text
`inventory.txt`	`inventory`	Pipe-delimited text
`projects.txt`	`projects`	Pipe-delimited text
`gldetails.csv`	`glmaster`, `gldetails`, `glsummary`	Sage 50 G/L detail export (CSV)
`openingbalances.csv`	`glmaster.opening_balance`	CSV with account number and opening balance

13.2 Sage 50 G/L Import

The `gldetails.csv` file is a direct export from Sage 50 Simply Accounting. The import script:

1. Creates account records in `glmaster` if they do not already exist.
2. Inserts transaction history into `gldetails`.
3. Rebuilds `glsummary` aggregations (debit/credit totals per account per period).

> **Warning:** Re-running the script will attempt to insert duplicate rows. It should only be run once on a fresh database.

---

14. Database Design

14.1 Table Inventory

Category	Tables
General Ledger	`glmaster`, `gldetails`, `glsummary`
Invoicing	`invoicemaster`, `invoicedetails`, `invoice_attachments`
Cash Receipts	`cashreceipts`
Journal	`journalmaster`, `journaldetails`
Timesheets	`timesheetmaster`, `timesheetdetail`, `timesheetworkitem`
Lookups	`customers`, `inventory`, `projects`, `salesmen`, `taxcodes`, `payment_terms`, `country`, `provinces`
System	`settings`, `users`, `useraccess`, `todos`

14.2 Key Relationships

```
customers ──────────────────────┐
                                ├──► invoicemaster ──► invoicedetails
salesmen ───────────────────────┤                  └──► invoice_attachments
projects ───────────────────────┘
                                    invoicemaster ──► cashreceipts

glmaster ──► gldetails ◄── journaldetails (via account_number)
          └──► glsummary

salesmen ──► timesheetmaster ──► timesheetdetail ──► timesheetworkitem
customers ──► timesheetmaster (customer_id FK)

users ──► useraccess
users ──► todos
```

14.3 Settings Table

All runtime configuration is stored as key-value pairs in the `settings` table (`setting_key`, `setting_value`). Defaults are seeded by `db.ensure_app_tables()` on first launch.

Key settings:

Key	Default	Purpose
`company_name`	`My Company`	Used on invoices and reports
`fiscal_year_start` / `fiscal_year_end`	`2024-01-01` / `2024-12-31`	Default report date range
`tax_rate`	`13.00`	Ontario HST rate applied to invoices
`currency`	`CAD`	Display currency
`ar_account`	(empty)	G/L account number for Accounts Receivable
`revenue_account`	(empty)	G/L account number for Revenue
`tax_account`	(empty)	G/L account number for Tax Payable
`cash_account`	(empty)	G/L account number for Bank/Cash
`invoice_prefix`	`INV-`	Prefix for invoice auto-numbers
`next_invoice_number`	`1001`	Next invoice sequence number
`next_journal_number`	`1001`	Next journal entry sequence number
`next_receipt_number`	`1001`	Next cash receipt sequence number
`receipt_prefix`	`CR-`	Prefix for cash receipt auto-numbers
`timesheet_require_approval`	`0`	If `1`, timesheet emails include an approval request
`smtp_host` / `smtp_port`	(configured)	Outbound email server

14.4 Auto-Number Sequences

Sequences are managed directly in the `settings` table. Each time a new invoice, journal entry, or cash receipt is saved, the corresponding `next_*` key is incremented by 1 within the same transaction.

---

15. Business Rules and Validations

15.1 General

- All monetary values are stored as `DECIMAL(18,2)`.
- Dates are stored as SQL `DATE`; the UI uses `MM/DD/YYYY` format throughout.
- Empty or null amounts are treated as `0.00`.

15.2 Invoicing

- An invoice must have at least one line item of type `item` before it can be saved.
- Tax is calculated as `subtotal × (tax_rate / 100)` and stored separately as `tax_amount`.
- `total = subtotal + tax_amount`.
- Invoices with status `Void` are excluded from AR balance calculations.

15.3 Cash Receipts

- A receipt must have a customer, date, amount > 0, bank account, and AR account before it can be posted.
- Once posted, a cash receipt cannot be edited or deleted.
- Posting is final and immediately updates `gldetails` and `glsummary`.

15.4 Journal Entries

- Entries must be balanced (`Σ debits = Σ credits`) before posting.
- At least two detail lines are required.
- Only `Draft` entries can be edited or deleted; `Posted` entries are immutable.

15.5 Timesheets

- Only one timesheet per salesman per calendar month is allowed (`UNIQUE` constraint on `(salesman_id, ts_year, ts_month)`).
- Only `Draft` and `Rejected` timesheets can be edited.
- `Approved` timesheets are immutable.
- Total hours must be ≥ 0; the system does not enforce a maximum.

15.6 Chart of Accounts

- Account numbers must be unique in `glmaster`.
- Accounts referenced by active `gldetails` rows cannot be deleted.

---

16. Integrations and Output Formats

16.1 Email (SMTP)

All outbound email uses Python's `smtplib` with optional TLS (`STARTTLS`). SMTP credentials are stored in the `settings` table. Three email types are supported:

Email Type	Trigger	Attachment
Invoice delivery	"Email Invoice" button in Invoicing module	PDF invoice
Timesheet delivery	"Send Timesheet to Client" button	PDF timesheet
Support request	"Support" button in main toolbar	None
New user welcome	User created in Configuration → Users	None

16.2 PDF Generation (ReportLab)

ReportLab `platypus` is used to produce all PDF documents:

Document	Orientation	Key Content
Invoice	Portrait	Company header, logo, line items, tax, total
Timesheet	Landscape	Consultant info, daily grid, work items, total hours
Trial Balance	Portrait	Account list with debit/credit balances, grand totals
Income Statement	Portrait	Revenue, expense, and net income sections
Balance Sheet	Portrait	Assets, liabilities, equity sections
Chart of Accounts	Portrait	Account number, name, type, opening balance, net balance

16.3 Excel Export (openpyxl)

Excel exports are offered from the G/L module, Reports module, and Timesheet module. Workbooks use styled headers (`PatternFill`, `Font`, `Alignment`, `Border`) and appropriate column widths.

16.4 CSV Export

The G/L module exports the Chart of Accounts to CSV. The Journal module exports journal entries to CSV. The `exportdir/accountant.csv` file is the standard G/L summary export used by the accountant.

17. Build and Deployment

17.1 Running from Source

```
pip install pyodbc reportlab openpyxl tkcalendar pdfplumber
python app.py
```

17.2 Standalone Executable (PyInstaller)

Run `build.bat` to produce a self-contained executable:

```
dist\MyAccounting\MyAccounting.exe
```

The build process:

1. Upgrades pip and installs all dependencies.
2. Cleans previous `build/` and `dist/` directories.
3. Runs PyInstaller with `MyAccounting.spec`.
4. Copies `importdir/` and `exportdir/` into the distribution folder.

Key spec settings:

- Tcl/Tk 8.6.12 DLLs are pinned from the Anaconda environment to prevent version-mismatch errors.
- `rthook_tcl.py` sets `TCL_LIBRARY` and `TK_LIBRARY` at frozen-app startup.
- Large Anaconda packages (`scipy`, `pandas`, `matplotlib`, etc.) are excluded.
- `app.json` is bundled as a data file.

17.3 Target Machine Requirements

Requirement	Details
OS	Windows 10 or later
SQL Server	Local instance, database `myaccounting`, Windows Authentication
ODBC Driver	ODBC Driver 17 for SQL Server
Python (source only)	3.9 or later

17.4 First-Run Setup

1. Create the `myaccounting` database on the local SQL Server instance.
2. Run `python import_data.py` once to load seed data.
3. Launch `app.py` (or `MyAccounting.exe`). All remaining tables are created automatically.
4. Log in with `admin` / `admin` and change the password via **Configuration** → **Users**.
5. Configure company info, fiscal year, G/L account mappings, and SMTP in **Configuration**.

---

18. Security Considerations

Area	Control
Password storage	PBKDF2-HMAC SHA-256, 260,000 iterations, random 16-byte salt per password
Password comparison	`hmac.compare_digest` — constant-time comparison prevents timing attacks
SQL injection	All database queries use parameterised statements (`pyodbc` `?` placeholders)
Authentication	Session is established once at login; the user context dict is held in memory for the session lifetime
Access control	Module-level access enforced in each module on load; admin role bypasses `useraccess` table
SMTP credentials	Stored in the `settings` table in the local SQL Server database; not exposed in source files in published builds
Network exposure	No network listener; the application is a desktop client connecting to a local SQL Server via ODBC
Audit trail	Journal entries record `created_date` and `posted_date`; cash receipts and invoices record creation timestamps

---

19. Application Forms and Dialogs

This section catalogues every modal dialog in the application, grouped by module. Each entry shows the fields presented to the user, their widget type, accepted values, and any validation rules enforced on save.

19.1 Authentication

LoginDialog (`login_dialog.py`)

Trigger: Displayed automatically at application startup before the main window is shown.

Field	Widget	Notes
Username	Entry	Plain text
Password	Entry	Masked (`show="*"`)

Actions: Sign In, Cancel
Validation: Both fields required; credentials verified against the `users` table using PBKDF2-HMAC.

19.2 Application-Level Dialogs (`app.py`)

_AboutDialog

Trigger: Help → About menu item.

Field	Widget	Notes
App Name	Label (read-only)	From `app.json`
Version	Label (read-only)	From `app.json`
Release Date	Label (read-only)	From `app.json`, formatted as "Month DD, YYYY"
Description	Label (read-only)	From `app.json`
Author	Label (read-only)	From `app.json`

Actions: Close
Validation: None (informational only).

_SupportDialog

Trigger: Help → Support Request menu item.

Field	Widget	Notes
Your Name	Entry	Free text
Contact Info (email / phone)	Entry	Free text
Type of Problem	Combobox (readonly)	Bug / Error, Question, Feature Request, Performance Issue, Other
Description	Text (8 rows, scrollable)	Free text

Actions: Submit, Cancel
Validation: Name and Description are required. SMTP settings must be configured; warns if not.

Side-effect: Sends an email to the configured support address via the SMTP settings.

_TodoDialog

Trigger: Dashboard → Add To-Do button, or double-click an existing to-do item.

Field	Widget	Notes
Title	Entry	Free text
Notes	Text (4 rows)	Free text, optional
Due Date	Entry + calendar button	MM/DD/YYYY format; calendar popup available
Priority	Combobox (readonly)	Low, Normal (default), High
Status	Combobox (readonly)	Open (default), In Progress, Done

Actions: Save, Cancel
Validation: Title required. Due Date must be MM/DD/YYYY when provided.

19.3 Configuration Module (`config_module.py`)

Company Info tab (ConfigModule)

Trigger: Configuration → Company Info tab (settings panel, not a modal dialog).

Field	Widget	Notes
Company Name	Entry	
Address Line 1	Entry	
Address Line 2	Entry	
City	Entry	
Province	Entry	
Postal Code	Entry	
Country	Entry	
Phone	Entry	
Fax	Entry	
Email	Entry	
Website	Entry	
Logo Path	Entry + Browse button	File path to a PNG/JPG image

Actions: Save Company Info
Validation: None enforced at field level; all fields saved as-is to the `settings` table.

Fiscal Year & Settings tab (ConfigModule)

Trigger: Configuration → Fiscal Year & Settings tab.

Field	Widget	Notes
Fiscal Year Start	Entry	YYYY-MM-DD
Fiscal Year End	Entry	YYYY-MM-DD
Tax Rate (%)	Entry	Numeric
Currency	Entry	3-letter code, e.g. CAD
Invoice Prefix	Entry	e.g. "INV-"
Next Invoice #	Entry	Auto-incremented integer
Next Journal #	Entry	Auto-incremented integer
A/R Account #	Combobox	Populated from GL master
Revenue Account #	Combobox	Populated from GL master
Tax Account #	Combobox	Populated from GL master
Cash Account #	Combobox	Populated from GL master

Actions: Save Settings

Email Settings tab (ConfigModule)

Trigger: Configuration → Email Settings tab.

Field	Widget	Notes
SMTP Host	Entry	e.g. smtp.gmail.com
SMTP Port	Entry	e.g. 587
SMTP User	Entry	Sender email address
SMTP Password	Entry (masked)	Stored in `settings` table
Use TLS (1/0)	Entry	"1" = TLS enabled
Support Email	Entry	Destination for support requests
Invoice Email Subject	Entry	Supports `{invoice_number}`, `{customer}`, `{total}`, `{company}`, `{date}` placeholders
Invoice Email Body	Text (8 rows)	Same placeholders available; hint shown below the field

Actions: Save Email Settings

Timesheet Settings tab (ConfigModule)

Trigger: Configuration → Timesheet tab.

Field	Widget	Notes
Default Shift	Combobox (readonly)	"8 to 4 — 08:00–12:00 / 12:30–16:00 (7.5 hrs)" or "9 to 5 — 09:00–12:00 / 12:30–17:00 (7.5 hrs)"
Require client approval	Checkbutton	When checked, approval request is included in timesheet email

Actions: Save Timesheet Settings

CustomerDialog

Trigger: Configuration → Customers → Add Customer / Edit Customer button.

Tab 1 — General

Field	Widget	Notes
Company	Entry	Required
Contact	Entry	
Address 1	Entry	
Address 2	Entry	
City	Entry	
Country	Combobox (readonly)	Populated from `country` table; auto-fills Currency
Province	Combobox (readonly)	Filtered by selected Country
Postal	Entry	Validated on focus-out: Canadian A1A 1A1, US 12345 or 12345-6789
Phone	Entry	Validated on focus-out: (###) ###-####
Cell	Entry	Validated on focus-out: (###) ###-####
Fax	Entry	Validated on focus-out: (###) ###-####
Email	Entry	
Website	Entry + globe button	Button opens URL in browser
Currency	Entry (disabled)	Auto-filled when Country is selected
Type	Entry	Customer type label
Client Manager	*LabelFrame section*	
— Name	Entry	
— Phone	Entry	
— Email	Entry	

Tab 2 — Ship-To

Field	Widget	Notes
Same as billing address	Checkbutton	When checked, copies billing address and disables all Ship-To fields
Country	Combobox (readonly)	Filtered from `country` table
Province/State	Combobox (readonly)	Filtered by Ship-To Country
Address 1	Entry	
Address 2	Entry	
City	Entry	
Postal/ZIP	Entry	Validated on focus-out

Tab 3 — Settings

Field	Widget	Notes
Salesman	Combobox (readonly)	ID – Name display from `salesmen` table
Revenue Account #	Combobox (readonly)	Account# – Name display from `glmaster`
Tax Code	Combobox (readonly)	Code – Name display from `taxcodes`
Credit Limit	Entry	Numeric
Payment Terms	Combobox (readonly)	ID – Name from `payment_terms`
Language	Combobox (readonly)	en, fr
Invoice Delivery	Combobox (readonly)	email, mail, fax, none
Notes	Text (4 rows)	Free text

Actions: Save, Cancel
Validation: Company name required.

InventoryDialog

Trigger: Configuration → Inventory → Add Item / Edit Item.

Field	Widget	Notes
Item Code	Entry	Required; unique identifier
Description	Entry	
Type	Entry	e.g. “Service”, “Product”
Unit	Entry	e.g. “hrs”, “each”
Price	Entry	Numeric; formatted as currency
Cost	Entry	Numeric; formatted as currency
GL Account	Entry	Account number (not a combobox)

Actions: Save, Cancel
Validation: Item Code required.

ProjectDialog

Trigger: Configuration → Projects → Add Project / Edit Project.

Field	Widget	Notes
Project Code	Entry	Required; unique identifier
Start Date	Entry	MM/DD/YYYY

Actions: Save, Cancel
Validation: Project Code required.

SalesmanDialog

Trigger: Configuration → Salesmen → Add Salesman / Edit Salesman.

Field	Widget	Notes
Name	Entry	Required
Signature Image	Entry + Browse button	File path; used on printed invoices/timesheets

Actions: Save, Cancel
Validation: Name required.

TaxCodeDialog

Trigger: Configuration → Tax Codes → Add Tax Code / Edit Tax Code.

Field	Widget	Notes
Code (max 10)	Entry	Disabled on edit (primary key)
Name	Entry	
Tax Components (sub-grid)	Treeview	Seq, Name, Rate (%), Tax Account

Treeview buttons: Add Component, Edit Component, Delete Component (each opens `_TaxDetailRowDialog`)

Actions: Save, Cancel
Validation: Code required; code is locked after creation.

_TaxDetailRowDialog

Trigger: Add Component or Edit Component button inside `TaxCodeDialog`.

Field	Widget	Notes
Name (e.g. HST, GST)	Entry	Required
Rate (%)	Entry	Numeric; e.g. 13 for 13%
Tax Account	Combobox	Account# – Name from `glmaster`

Actions: OK, Cancel
Validation: Name required; Rate must be numeric.

CountryDialog

Trigger: Configuration → Countries → Add Country / Edit Country.

Field	Widget	Notes
Code (2–3 chars)	Entry	Disabled on edit (primary key); auto-uppercased
Name	Entry	Required
Currency	Entry	3-letter code; auto-uppercased

Actions: Save, Cancel
Validation: All three fields required.

ProvinceDialog

Trigger: Configuration → Provinces → Add Province / Edit Province.

Field	Widget	Notes
Country	Combobox (readonly)	Code – Name from `country`; disabled on edit
Code (max 10)	Entry	Disabled on edit; auto-uppercased
Name	Entry	Required

Actions: Save, Cancel
Validation: Country, Code, and Name all required.

PaymentTermsDialog

Trigger: Configuration → Payment Terms → Add / Edit.

Field	Widget	Notes
Name	Entry	Required; e.g. “Net 30”

Actions: Save, Cancel
Validation: Name required.

UserDialog

Trigger: Configuration → Users (admin only) → Add User / Edit User.

Field	Widget	Notes
Username	Entry	Required
Full Name	Entry	Required
Email	Entry	
Role	Combobox (readonly)	admin, accountant
Password *(add only)*	Entry (masked)	Required for new users only
Confirm Password *(add only)*	Entry (masked)	Must match Password

Actions: Save, Cancel
Validation: Username and Full Name required. For new users: Password required and both password fields must match.

ChangePasswordDialog

Trigger: Admin clicking “Change Password” on a selected user in the Users panel.

Field	Widget	Notes
New Password	Entry (masked)	Required
Confirm	Entry (masked)	Must match New Password

Actions: Save, Cancel
Validation: Password required; both fields must match.

19.4 Invoicing Module (`invoice_module.py`)

InvoiceDialog

Trigger: Invoicing → New Invoice button, or double-click an existing invoice.

Header fields

Field	Widget	Notes
Invoice #	Entry	Auto-assigned from settings `next_invoice_number`
Date	Entry + calendar button	MM/DD/YYYY
Status	Combobox (readonly)	Draft, Sent, Paid, Cancelled, Posted
Customer	Combobox	Filtered from `customers` table
Project	Combobox	Filtered from `projects` table
Salesman	Combobox	Filtered from `salesmen` table

Line items area — Treeview with columns: #, Item Code, Description, Qty, Unit Price, Amount

Line item buttons: Add Line Item (opens `InvoiceLineDialog`), Add Comment (opens `InvoiceCommentDialog`), Edit (opens respective dialog), Delete

Totals display: Subtotal, Tax (calculated from customer tax code), Total

Actions: Save, Cancel, Print PDF, Email, Attachments (opens `InvoiceAttachmentsDialog`)
Validation: Customer required; at least one line item required to post.

InvoiceLineDialog

Trigger: Add Line Item button or Edit on a non-comment line in `InvoiceDialog`.

Field	Widget	Notes
Item	Combobox (filterable)	Type to narrow; populated from `inventory`
Description	Entry	Auto-filled from item; editable
Quantity	Entry	Numeric
Unit Price	Entry	Numeric; auto-filled from inventory price
Amount	Entry (read-only)	Auto-calculated: Qty × Unit Price

Actions: Save, Cancel
Validation: Item, Quantity, and Unit Price required; Amount is computed automatically.

InvoiceCommentDialog

Trigger: Add Comment button in `InvoiceDialog`.

Field	Widget	Notes
Comment	Entry	Free text

Actions: OK, Cancel
Validation: Comment must not be empty.

InvoiceAttachmentsDialog

Trigger: Attachments button in `InvoiceDialog`.

Control	Type	Notes
File list	Treeview	Columns: Filename, Full Path
Add Attachment	Button	Opens file browser; adds selected file to list
Delete Attachment	Button	Removes selected entry

Actions: OK, Cancel
Validation: None beyond file existence on add.

PrinterSelectDialog

Trigger: Print PDF → select printer (when multiple printers are available).

Control	Type	Notes
Printer list	Listbox	System printers enumerated via `win32print`

Actions: Select, Cancel

19.5 Cash Receipts Module (`cash_receipt_module.py`)

CashReceiptDialog

Trigger: Cash Receipts → New Receipt button, or double-click an existing receipt.

Field	Widget	Notes
Receipt #	Entry	Auto-assigned sequence
Date	Entry + calendar button	MM/DD/YYYY
Customer	Combobox	Populated from `customers`
Apply to Invoice	Combobox	Populated dynamically from open invoices for the selected customer
Amount	Entry	Numeric
Bank Account #	Entry	GL account number for deposit
Memo	Entry	Free text

Actions: Save, Cancel, Post
Validation: Customer, Amount and Bank Account required. Post transitions status and creates GL entries.

19.6 General Journal Module (`journal_module.py`)

JournalDialog

Trigger: Journal → New Journal Entry button, or double-click an existing entry.

Header fields

Field	Widget	Notes
Entry #	Entry	Auto-assigned from `next_journal_number`
Date	Entry + calendar button	MM/DD/YYYY
Description	Entry	Free text
Reference	Entry	Optional reference number

Lines area — Treeview with columns: #, Account #, Account Name, Description, Debit, Credit

Line buttons: Add Line (opens `LineDialog`), Edit Line, Delete Line

Totals bar: Total Debits, Total Credits, balance status label ("BALANCED" in green or "OUT OF BALANCE" in red)

Actions: Save, Cancel
Validation: Entry must be balanced (Total Debits = Total Credits) before saving.

LineDialog

Trigger: Add Line or Edit Line button in `JournalDialog`.

Field	Widget	Notes
Account	Combobox (filterable)	Type to filter; populated from `glmaster`
Description	Entry	Free text
Debit	Entry	Numeric; leave at 0 for credit line
Credit	Entry	Numeric; leave at 0 for debit line

Actions: OK, Cancel
Validation: Account required; exactly one of Debit or Credit must be non-zero.

_ExpenseHeaderDialog

Trigger: Initiated during CSV GL expense import process (`import_data.py`).

Collects the journal entry header fields (Date, Description, Reference) before the import maps CSV rows to debit/credit lines.

Field	Widget	Notes
Date	Entry	MM/DD/YYYY
Description	Entry	Default filled from import filename
Reference	Entry	Optional

Actions: OK, Cancel

19.7 Chart of Accounts Module (`gl_module.py`)

AccountDialog

Trigger: GL → Add Account button, or Edit Account on a selected account.

Field	Widget	Notes
Account Number	Entry	Disabled on edit (primary key)
Account Name	Entry	Required
Opening Balance	Entry	Numeric; defaults to 0.00
Balance Type	Combobox (readonly)	Dr (debit-normal) or Cr (credit-normal)

Actions: Save, Cancel
Validation: Account Number and Account Name required. Duplicate account number check on insert.

TransactionDialog

Trigger: Double-click an account row in the GL module, or View Transactions button.

Control	Type	Notes
Account # header	Label	Displays selected account
Transaction list	Treeview (read-only)	Date, Description, Source, Trans #, Debit, Credit, Bal Type; loaded from `gldetails`
Total line	Label	Running debit / credit totals shown below the treeview

Actions: Close
Validation: None (read-only viewer).

ExportFormatDialog

Trigger: GL → Export Chart of Accounts button.

Control	Type	Notes
CSV button	Button	Exports to a CSV file
Excel button	Button	Exports to an `.xlsx` file via `openpyxl`
PDF button	Button	Exports to a PDF file via `reportlab`

Actions: CSV, Excel, PDF, Cancel
Validation: None; format selection triggers the export immediately.

19.8 Timesheet Module (`timesheet_module.py`)

_DetailDialog (Day Entry)

Trigger: Double-click a day cell in the weekly timesheet grid, or click a date cell to open the day's time detail.

Field	Widget	Notes
Date	Label (read-only)	Identifies the day being edited
AM Start	Entry	HH:MM format
AM End	Entry	HH:MM format
PM Start	Entry	HH:MM format
PM End	Entry	HH:MM format
Other Start	Entry	HH:MM for overtime or alternate periods
Other End	Entry	HH:MM
Total Hours	Label (computed)	Calculated from the six time fields

Actions: OK, Cancel
Validation: Time fields must be parseable as HH:MM.

Work Item Dialog (anonymous inline class)

Trigger: Timesheet → Add Work Item button, or double-click an existing work item.

Field	Widget	Notes
Description	Text (4 rows + scrollbar)	Free text description of the work performed
DevOps Work Item #	Entry	Optional numeric reference
Hours	Entry	Numeric; hours spent on this task

Actions: OK, Cancel
Validation: Description required; Hours must be numeric when provided.

_TimesheetImportPreviewDialog

Trigger: Timesheet → Import from TPG PDF/CSV button after the file is parsed.

Control	Type	Notes
Preview list	Treeview	Parsed rows: Date, AM Start, AM End, PM Start, PM End, Other hours
Row count summary	Label	Shows number of rows to be imported

Actions: Confirm Import, Cancel
Validation: Warns if any rows conflict with existing entries.

_CustomerPickerDialog

Trigger: Timesheet → Email Timesheet button when the customer must be selected.

Control	Type	Notes
Search box	Entry	Filters the customer list live
Customer list	Treeview / Listbox	ID, Company name

Actions: Select, Cancel
Validation: A customer must be selected before proceeding.

19.9 Form Summary

Module	Dialog Class	Purpose
Auth	`LoginDialog`	Application sign-in
App	`_AboutDialog`	Version / info display
App	`_SupportDialog`	Send support request by email
App	`_TodoDialog`	Create or edit a dashboard to-do item
Config	`CustomerDialog`	Add / edit customer (3-tab, 32+ fields)
Config	`InventoryDialog`	Add / edit inventory item
Config	`ProjectDialog`	Add / edit project
Config	`SalesmanDialog`	Add / edit salesman
Config	`TaxCodeDialog`	Add / edit tax code with component rows
Config	`_TaxDetailRowDialog`	Add / edit one tax component
Config	`CountryDialog`	Add / edit country
Config	`ProvinceDialog`	Add / edit province or state
Config	`PaymentTermsDialog`	Add / edit a payment term
Config	`UserDialog`	Add / edit a user account
Config	`ChangePasswordDialog`	Reset a user's password
Invoicing	`InvoiceDialog`	Full invoice editor
Invoicing	`InvoiceLineDialog`	Add / edit invoice line item
Invoicing	`InvoiceCommentDialog`	Add a comment line to an invoice
Invoicing	`InvoiceAttachmentsDialog`	Manage file attachments on an invoice
Invoicing	`PrinterSelectDialog`	Choose printer for PDF printing
Cash Receipts	`CashReceiptDialog`	Enter / edit a cash receipt
Journal	`JournalDialog`	Full journal entry editor
Journal	`LineDialog`	Add / edit one debit or credit line
Journal	`_ExpenseHeaderDialog`	Header for GL CSV expense import
GL	`AccountDialog`	Add / edit a GL account
GL	`TransactionDialog`	Read-only transaction history viewer
GL	`ExportFormatDialog`	Choose CSV / Excel / PDF export format

Module	Dialog Class	Purpose
Timesheet	`_DetailDialog`	Enter daily AM / PM / Other times
Timesheet	Work Item dialog	Add / edit a work item entry
Timesheet	`_TimesheetImportPreviewDialog`	Preview rows before TPG import
Timesheet	`_CustomerPickerDialog`	Select customer for timesheet email

End of Functional Analysis